

Meeting Notes – Project Scope, Core Workflow & Requirements

1. Overall Project Priority

Hunter emphasised that the **primary goal is to build the annotation infrastructure/platform**, rather than building or fine-tuning AI models themselves.

The main focus should be on making the **end-to-end annotation workflow work reliably**, regardless of which AI model is eventually used.

Key point

- We **do not need to fine-tune or implement an actual AI model** as part of the core deliverable.
- The AI annotation step can initially be **simulated using fixtures/mock APIs**.
- The priority is to demonstrate that the platform can:
 - Trigger an AI annotation process.
 - Receive/simulate the annotated result.
 - Move the result into the human review workflow.
 - Handle multiple reviewers.
 - Handle disagreements/disputes.
 - Produce a final judgement/reconciled result.

Hunter described this as the **critical path / essential workflow** that the project must achieve.

2. Critical / Must-Have Workflow

The essential workflow should look approximately like:

Create Project → Configure Policy → Create Task → Add Items/Dataset → AI Annotation → Human Review → Cross-Validation → Dispute Resolution → Final Judgement/Submission

Step-by-step

1. Project creation

- User creates a new project.
- Project contains the overall scope/objective of the work.

2. Policy configuration

- Define how the project/task should be reviewed.
- Policy should determine things such as:
 - Number of human annotators/reviewers.
 - Number of review rounds.
 - Cross-validation requirements.
 - Percentage/rate of items that should be independently reviewed.

3. Task creation

- Create a task within the project.
- Tasks represent a particular purpose/scope of annotation or judgement.
- Different tasks can exist under the same project.

4. Items / dataset

- A task contains multiple items.
- Items are the actual data/artifacts being processed.
- Examples:
 - Images
 - Videos
 - Text
 - Other datasets/artifacts
- A task could potentially contain hundreds or thousands of items.

5. AI annotation

- AI performs the first-pass annotation.
- **For this project, the actual AI model can be mocked/simulated.**
- Fixtures/mock APIs can be used to return an already-annotated result.
- The purpose is to demonstrate the platform's workflow, not to build the model itself.

6. Human review

- AI-generated annotations then move into the human review process.
- Humans validate, correct, or provide feedback on the AI output.

7. Cross-validation

- Multiple people may review the same item.
- The policy determines:
 - How many reviewers are required.
 - How many review rounds are required.
 - What percentage of items should receive additional/cross-review.

Example given by Hunter:

- A project could define that **30% of tasks/items** should be independently reviewed by different people.

8. Dispute resolution

- If multiple reviewers provide different judgements, the item enters a dispute/reconciliation process.
- A higher-level user can step in to resolve the disagreement.
- An expert may provide the final decision/judgement.

9. Final submission/judgement

- Once review/dispute resolution is complete, the final result is submitted/reconciled.

3. Project → Task → Item Model

Hunter clarified the distinction between these three concepts.

Project

The **overall body of work / objective**.

Example:

- Autonomous Driving Project

Task

A specific scope or purpose within a project.

For example, within an autonomous driving project:

- Image annotation task.
- Video annotation task.
- Traffic-rule understanding task.
- Model performance evaluation task.

Tasks are not necessarily differentiated only by input modality. They represent a **specific purpose or goal** within the overall project.

Item

The actual data/artifact being processed inside a task.

Examples:

- Individual image.
- Individual video.
- Individual text input.

- Dataset entry.

Example:

Project: Autonomous Driving

→ **Task:** Traffic Rule Understanding

→ **Items:** 1,000 video recordings

Hunter stressed that this hierarchy should be considered when designing the system and lifecycle management.

4. Lifecycle Management

Hunter would like lifecycle management to eventually be supported for:

- Projects
- Tasks
- Items

These should have lifecycle states and management capabilities similar to one another.

Project lifecycle examples

- Created
- In progress / ongoing
- Completed
- Archived

Projects should also support management operations such as:

- Rename project.
- Update project.
- Update policy.
- Update team settings.
- Update storage location.
- Archive project.

Similar lifecycle thinking should be applied to **tasks and items**.

5. Annotation UI / Frontend

The annotation page **already exists**.

Hunter does **not** want us to integrate Label Studio directly.

Instead:

- The existing annotation page was **inspired/adapted from Label Studio**.
- Label Studio should be used as a reference for understanding how annotation information/UI can be organised.
- We should focus on making the existing annotation page **functional through the backend**.
- The goal is to make the annotation workflow work end-to-end rather than simply improving the UI.

Important clarification

Hunter specifically corrected the idea of “integrating Label Studio”:

We don’t need to integrate Label Studio. We should learn from how Label Studio organises the annotation UI and apply those ideas to our existing platform.

The broader vision is **not** to take Label Studio and simply add an AI/chat layer on top.

Instead, the goal is to build an **AI-assisted data annotation infrastructure** where AI can increasingly perform the first pass and humans can review/judge the results.

6. AI-Assisted Annotation Philosophy

The platform should be designed around the idea that:

AI can increasingly become the first-pass annotator, while humans provide review, judgement and validation.

Therefore, the platform should not be designed as a purely human annotation system.

However, implementing sophisticated real AI models is **not currently the priority**.

Initial implementation

Use:

- Fixtures
- Mock APIs

- Simulated AI responses

This allows us to demonstrate the complete workflow without spending project time on:

- Model training.
- Model fine-tuning.
- Building annotation models from scratch.

Future direction

Once the critical path is working and user acceptance criteria are satisfied, the system could be expanded to:

- Integrate real models.
- Preferably use open-source models.
- Allow users to bring their own model.
- Support self-hosted deployments.

The infrastructure should therefore be **model-agnostic and extensible**.

7. Task Types

Hunter mentioned that there are currently **two main task types**:

1. **Annotation**
2. **Judgement**

These should be treated as the two primary task categories.

For judgement tasks, the human is not simply providing a binary answer.

The system should support collecting:

- Feedback.
- Justification.
- Structured reasoning/reasoning logic.
- The actual judgement/decision.

The important idea is that the collected human feedback should be **traceable, verifiable and useful as a signal**, rather than just a simple “Yes/No” or “A/B” preference.

8. Human Judgement / Feedback

Hunter highlighted that human judgement is an important part of the platform.

The purpose of collecting human judgement is not limited to RLHF.

The broader objective is to capture **how humans reason and make decisions**, particularly in ambiguous or complex situations.

Examples of useful signals include:

- Why is an AI answer wrong?
- Why is one answer preferred over another?
- What reasoning led to the decision?
- What feedback should the AI/system learn from?
- How would a human approach a complex problem?
- How would a human break down a large problem?
- How would a human respond when an AI/agent encounters an unexpected situation?

Hunter specifically wants **structured reasoning and justification**, rather than only binary indicators.

9. Task Queue

The **task queue** is another major requirement.

Hunter described this as a classic distributed-system problem applied to the annotation context.

Example

A task may contain:

- 100 items.
- 1,000 items.
- Potentially much larger datasets.

When AI annotation is requested:

- Items need to be processed.
- The system needs to manage the queue.
- The AI annotation process should be handled reliably.
- Results need to move through the workflow after processing.

Engineering considerations

Hunter specifically mentioned:

- Large task queues.
- Retry handling.
- Failed jobs.
- Dead-letter handling / dead-letter queues.
- Stable processing.
- Reliable communication between frontend and backend.
- Reliable communication with AI/third-party services.

Hunter described this as a **classic distributed system design problem**, with the annotation platform providing the specific context.

10. Backend / Core Engine

One of the biggest gaps identified from the previous team's work was the **core backend/engine**.

The previous team reportedly did well on:

- Frontend/web surface.
- Schema design.
- Contracts.
- Thinking about integration.

However, the actual connection between:

platform → annotation engine → AI annotation process
was not sufficiently implemented.

Even a **simulated AI annotation workflow** was not completed to the level expected.

Therefore, our main engineering focus should be:

- Understand the existing backend.
- Understand the current schemas.
- Understand existing contracts.
- Diagnose what is already implemented.
- Design/complete the core engine.
- Connect the frontend workflow to the backend.
- Implement the simulated AI annotation workflow.
- Connect AI annotation results to human review.
- Implement review/cross-validation/dispute flow.
- Ensure the system is robust under larger workloads.

11. Schema & Contracts

Hunter strongly emphasised **schemas and contracts**.

There are essentially multiple contracts that need to be considered:

Frontend ↔ Backend

The frontend and backend need a clear contract defining:

- Request formats.
- Response formats.
- Data structures.
- Workflow states.
- Errors/failures.

Backend ↔ AI / Third-party services

There should also be a clear contract for communicating with:

- AI services.
- Mock AI services.
- Future third-party models/services.

Key expectation

The team should **first understand and freeze the required schema/contracts**, rather than immediately starting implementation.

Hunter believes a substantial portion of this work may already exist, so the first step should be to:

Inspect → Diagnose → Validate → Fix/Update → Implement

rather than redesigning everything from scratch.

12. Roles & Permissions

Hunter confirmed that roles have already been considered/implemented in the service/documentation.

The key roles mentioned are:

Annotator

- Performs annotation/review work on assigned items.

Reviewer

- Reviews annotations, particularly AI-generated annotations.
- Participates in cross-validation/review rounds.

Expert

- Steps in when reviewers disagree.
- Resolves disputes.
- Provides the final judgement where required.

Admin

- System/organisation-level administration.
- Manages organisation-level settings and users.

Hunter does **not** expect every possible role/rule to be implemented immediately.

Instead, the role/permission model should be designed in an **extensible/open-ended way**, so that additional roles and rules can be added later without major architectural changes.

13. Cross-Validation & Review Policy

Policies are an important part of the platform.

When creating a project/task, the user should be able to define things such as:

- Number of reviewers.
- Number of review rounds.
- Whether cross-validation is required.
- Percentage/rate of items requiring additional review.
- How disagreements should be handled.
- When an expert should be involved.

Example given by Hunter:

30% of items are independently reviewed by different reviewers.

If reviewers disagree during cross-validation:

- The disagreement becomes a dispute.
- A higher-level reviewer/expert resolves the dispute.
- The final judgement is recorded.

14. Organisation Management

Organisation management is considered **nice-to-have rather than urgent**.

Potential capabilities include:

- Creating/managing an organisation.
- Inviting users.
- Managing multiple accounts/users.
- Organisation-level administration.

Hunter clarified that an organisation can have multiple accounts/use cases/domains.

The important requirement is:

- A **unique organisation ID**.
- A **unique URL/identifier**.

The exact naming/ownership structure is less important at this stage.

Hunter referenced enterprise organisation-management systems as an example of the general concept.

15. Dashboard

A project dashboard is considered **nice-to-have**.

It does not need to be extremely comprehensive.

The basic dashboard should provide an overview of:

- Project status.
- Project progress.
- Organisation information.
- General project/task information.
- Relevant workflow statistics.

Hunter explicitly said that a basic dashboard is sufficient rather than spending significant time making it overly comprehensive.

16. Previous Team – Key Learnings

Hunter gave the previous team a generally **positive assessment**.

What the previous team did well

- Frontend/web interface.
- Overall web surface.
- Schema design.
- Contracts.
- Thinking about integration.
- Some aspects of system modelling.

Main gap

The biggest missing piece was the **actual backend/core engine and glue between the platform and AI annotation workflow**.

Hunter believes one reason was that the project did not start sufficiently from a **classic software engineering/system design perspective**.

There was significant attention on:

- Data models.
- Classes.
- Objects.
- Unified models.

But less emphasis was placed on the actual end-to-end operational flow of the system.

Main lesson for our team

We should avoid focusing only on:

- UI.
- Data models.
- Individual components.

Instead, we need to focus on the **complete working system**:

Frontend → Backend → Queue → AI/mock AI → Annotation result → Human review → Cross-validation → Dispute → Final judgement

17. Software Engineering Expectations

Hunter said the project will be assessed significantly from a **software engineering perspective**.

Important qualities include:

- Robustness.
- Stability.
- Reliability.
- Clear architecture.
- Good contracts.
- Scalable task processing.
- Retry mechanisms.
- Failure handling.
- Dead-letter handling.
- Extensibility.
- Clear separation between components.
- Ability to integrate future AI models.

The system should not just demonstrate that a happy-path workflow works once.

It should demonstrate that the infrastructure can **reliably handle the workflow as a real platform**.